

HW 6: Study Gini Index

Instructions

- Study Gini Index
- Compute the toy example using Gini Index
- Change criterion in the imported library, using Gini Index
- Compare Gini Index vs Entropy
- Use another dataset (datasets.csv)
- Play with parameters:
 - max_depth
 - min_samples_split
 - min_samples_leaf
- Explain your understanding after trying these different parameters

```
In [ ]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
In [158... toy = pd.read_csv('toy_data.csv')
```

```
In [182... def findginigain(x):
    results = []
    newdf = toy.groupby('buys computer').size()
    pyes = newdf['yes']/newdf.sum()
    pno = newdf['no']/newdf.sum()
    pargini = 1 - pyes**2 - pno**2
    list = toy.columns.drop('buys computer')
    for col in list:
        newdf = toy.groupby(col)['buys computer'].value_counts().unstack().fillna(0)
        newdf['n'] = newdf['no'] + newdf['yes']
        newdf['pyes'] = newdf['yes']/newdf['n']
        newdf['pno'] = newdf['no']/newdf['n']
        newdf['gini'] = 1 - newdf['pyes']**2 - newdf['pno']**2
        weight_gini = sum(newdf['n']/toy.value_counts().count()*newdf['gini'])
        gain_gini = pargini - weight_gini
        results.append({'Attribute': col, 'Gain Gini': gain_gini})

    return pd.DataFrame(results)
```

```
In [183... findginigain(toy)
```

Out[183...

	Attribute	Gain Gini
0	age	0.116327
1	income	0.018707
2	student	0.091837
3	credit rating	0.030612

```
In [77]: label_encoder = LabelEncoder()
toy['age'] = label_encoder.fit_transform(toy['age'])
toy['income'] = label_encoder.fit_transform(toy['income'])
toy['student'] = label_encoder.fit_transform(toy['student'])
toy['credit rating'] = label_encoder.fit_transform(toy['credit rating'])
toy['buys computer'] = label_encoder.fit_transform(toy['buys computer'])

X = toy.drop('buys computer', axis=1)
y = toy['buys computer']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

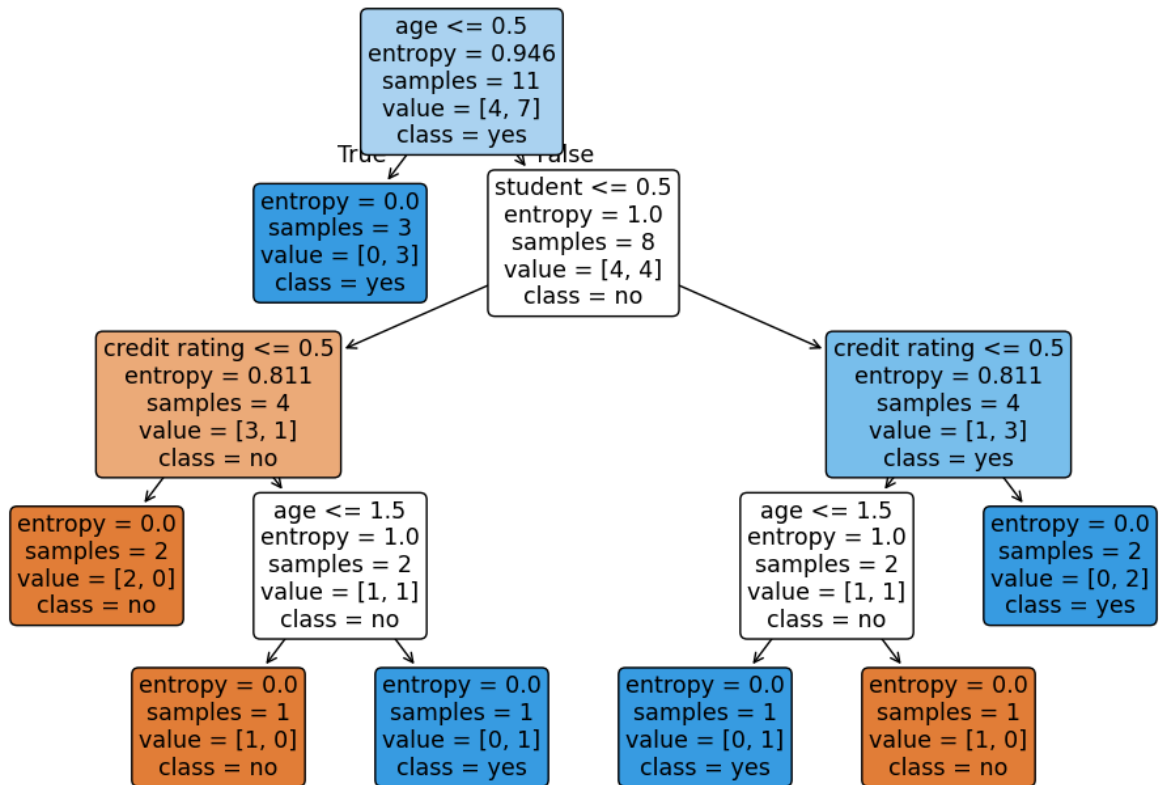
Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Confusion Matrix:

```
[[1 0]
 [0 2]]
```



```

In [78]: label_encoder = LabelEncoder()
toy['age'] = label_encoder.fit_transform(toy['age'])
toy['income'] = label_encoder.fit_transform(toy['income'])
toy['student'] = label_encoder.fit_transform(toy['student'])
toy['credit rating'] = label_encoder.fit_transform(toy['credit rating'])
toy['buys computer'] = label_encoder.fit_transform(toy['buys computer'])

X = toy.drop('buys computer', axis=1)
y = toy['buys computer']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion='gini', random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()

```

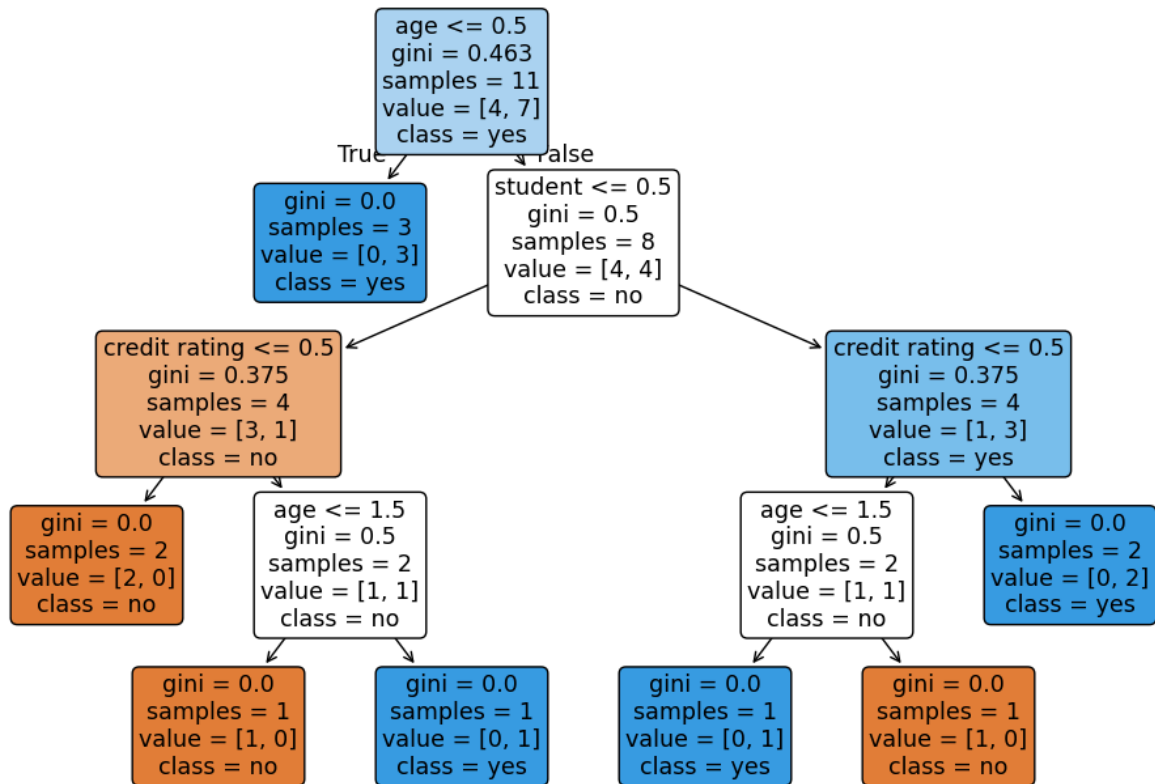
Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Confusion Matrix:

```
[[1 0]
 [0 2]]
```



```
In [81]: df = pd.read_csv('dataset.csv')
df
```

```
Out[81]:
```

	feature_0	feature_1	feature_2	feature_3	feature_4	target
0	0.374540	0.950714	0.731994	0.598658	0.156019	0
1	0.155995	0.058084	0.866176	0.601115	0.708073	1
2	0.020584	0.969910	0.832443	0.212339	0.181825	1
3	0.183405	0.304242	0.524756	0.431945	0.291229	0
4	0.611853	0.139494	0.292145	0.366362	0.456070	1
...	...	...	...	...	...	...
145	0.841829	0.139772	0.795267	0.201627	0.163656	1
146	0.164266	0.814575	0.665197	0.523065	0.358830	1
147	0.877201	0.392445	0.816599	0.439135	0.376944	1
148	0.462680	0.301378	0.747609	0.502720	0.232213	0
149	0.899575	0.383891	0.543553	0.906472	0.624238	1

150 rows × 6 columns

```
In [82]: X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

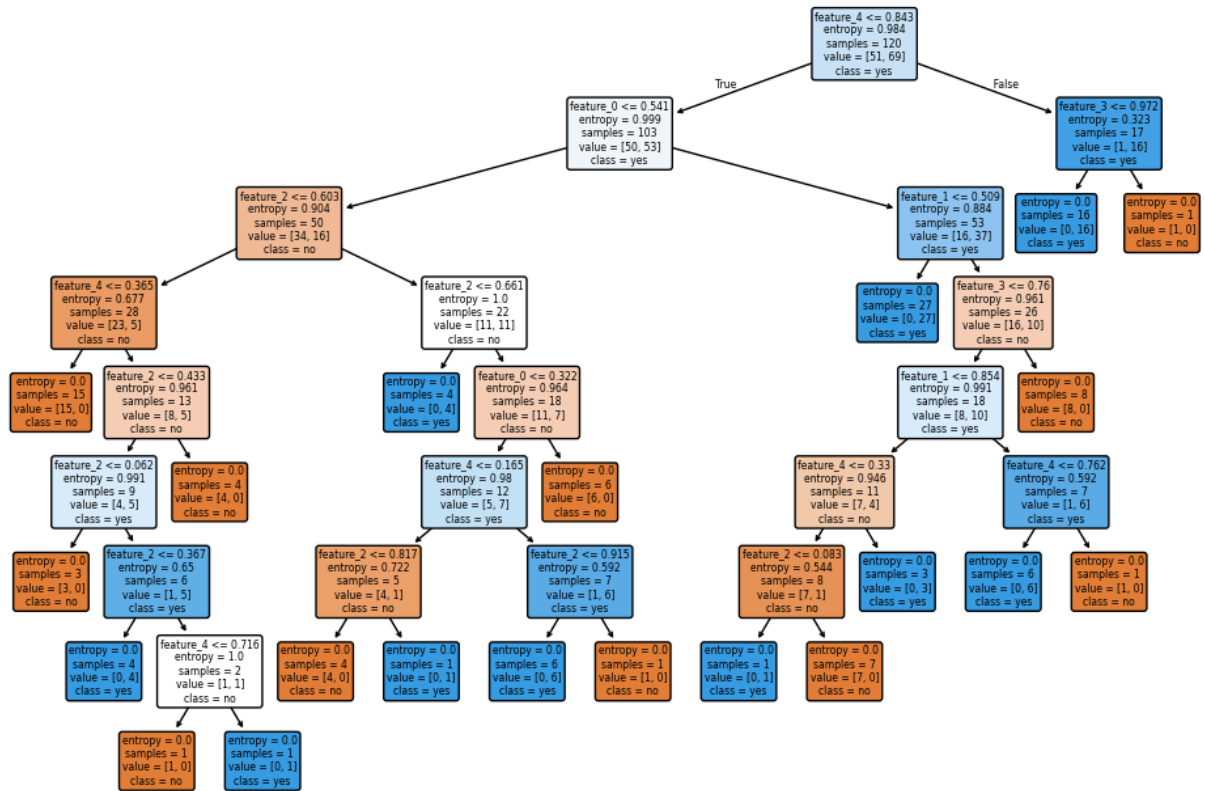
Accuracy: 0.67

Classification Report:

	precision	recall	f1-score	support
0	0.62	0.42	0.50	12
1	0.68	0.83	0.75	18
accuracy			0.67	30
macro avg	0.65	0.62	0.62	30
weighted avg	0.66	0.67	0.65	30

Confusion Matrix:

```
[[ 5  7]
 [ 3 15]]
```



```

In [83]: X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion='gini', random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()

```

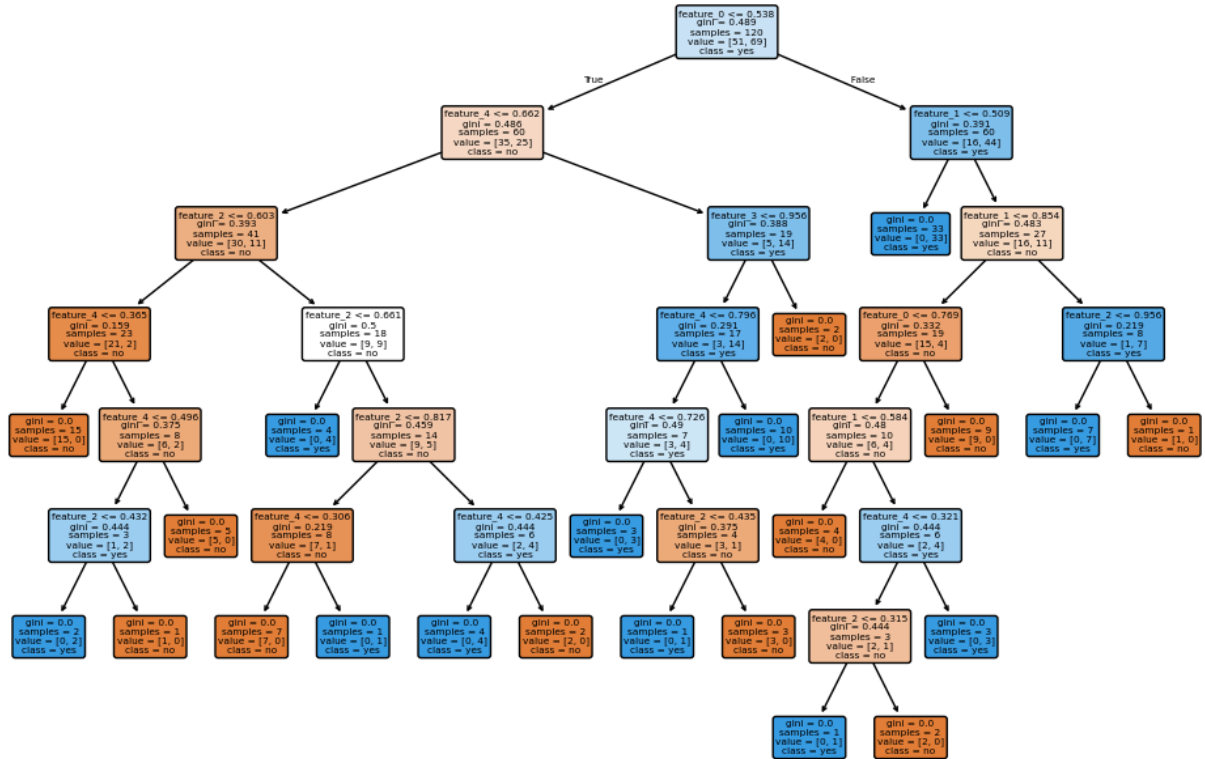
Accuracy: 0.80

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.75	0.75	12
1	0.83	0.83	0.83	18
accuracy			0.80	30
macro avg	0.79	0.79	0.79	30
weighted avg	0.80	0.80	0.80	30

Confusion Matrix:

```
[[ 9  3]
 [ 3 15]]
```



In [147...

```
clf = DecisionTreeClassifier(
    criterion='gini',
    max_depth=10,
    min_samples_split= 10,
    min_samples_leaf= 6,
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

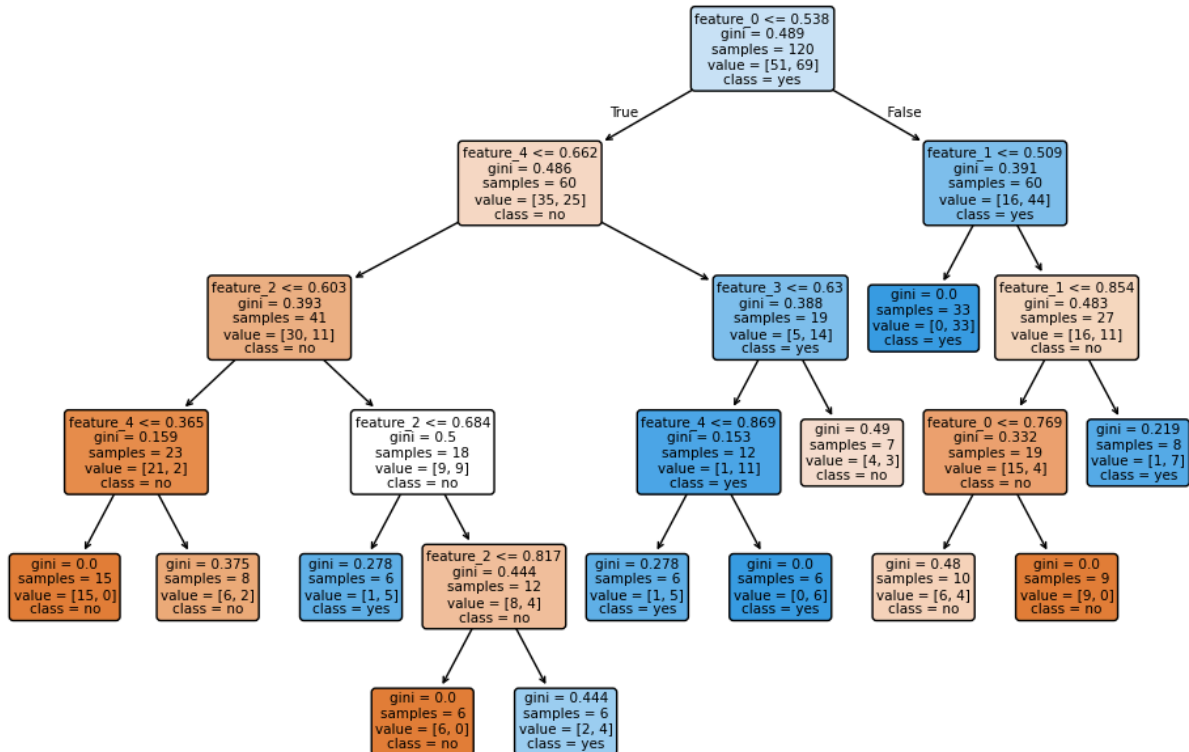
Accuracy: 0.83

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.83	0.80	12
1	0.88	0.83	0.86	18
accuracy			0.83	30
macro avg	0.83	0.83	0.83	30
weighted avg	0.84	0.83	0.83	30

Confusion Matrix:

```
[[10  2]
 [ 3 15]]
```



In [146...

```

clf = DecisionTreeClassifier(
    criterion='gini',
    max_depth=1,
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()

```

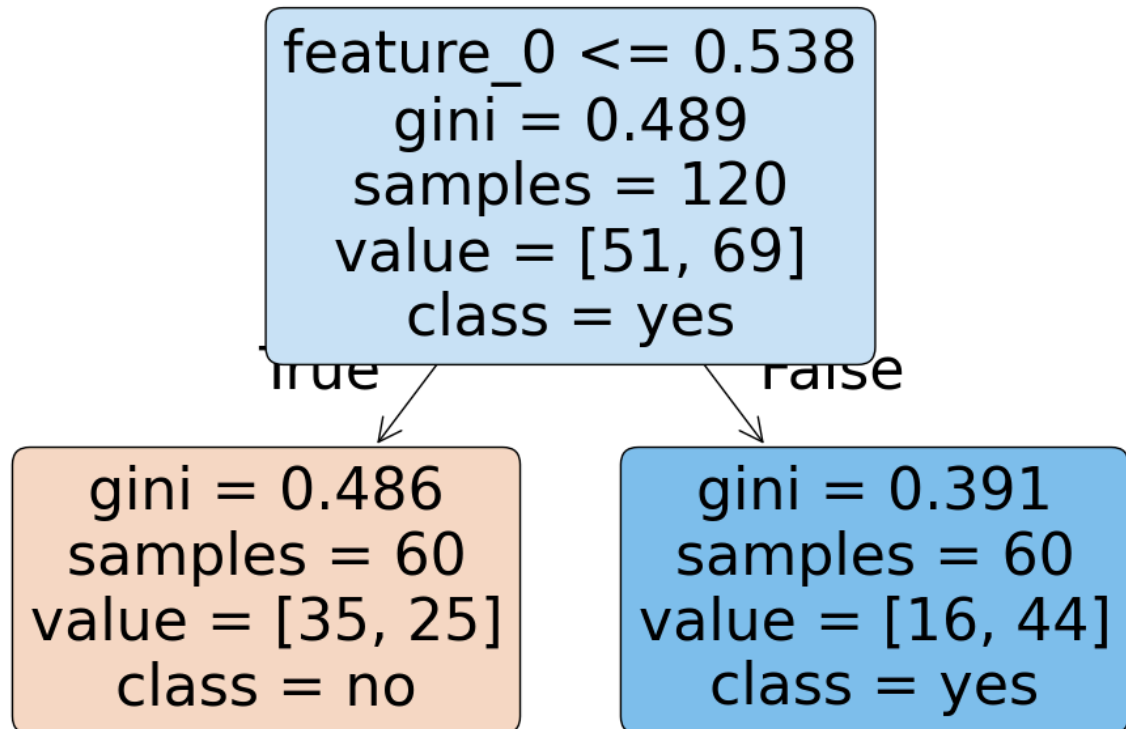
Accuracy: 0.57

Classification Report:

	precision	recall	f1-score	support
0	0.47	0.75	0.58	12
1	0.73	0.44	0.55	18
accuracy			0.57	30
macro avg	0.60	0.60	0.57	30
weighted avg	0.63	0.57	0.56	30

Confusion Matrix:

```
[[ 9  3]
 [10  8]]
```



```
In [151... clf = DecisionTreeClassifier(
    criterion='entropy',
    min_samples_split= 50,
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

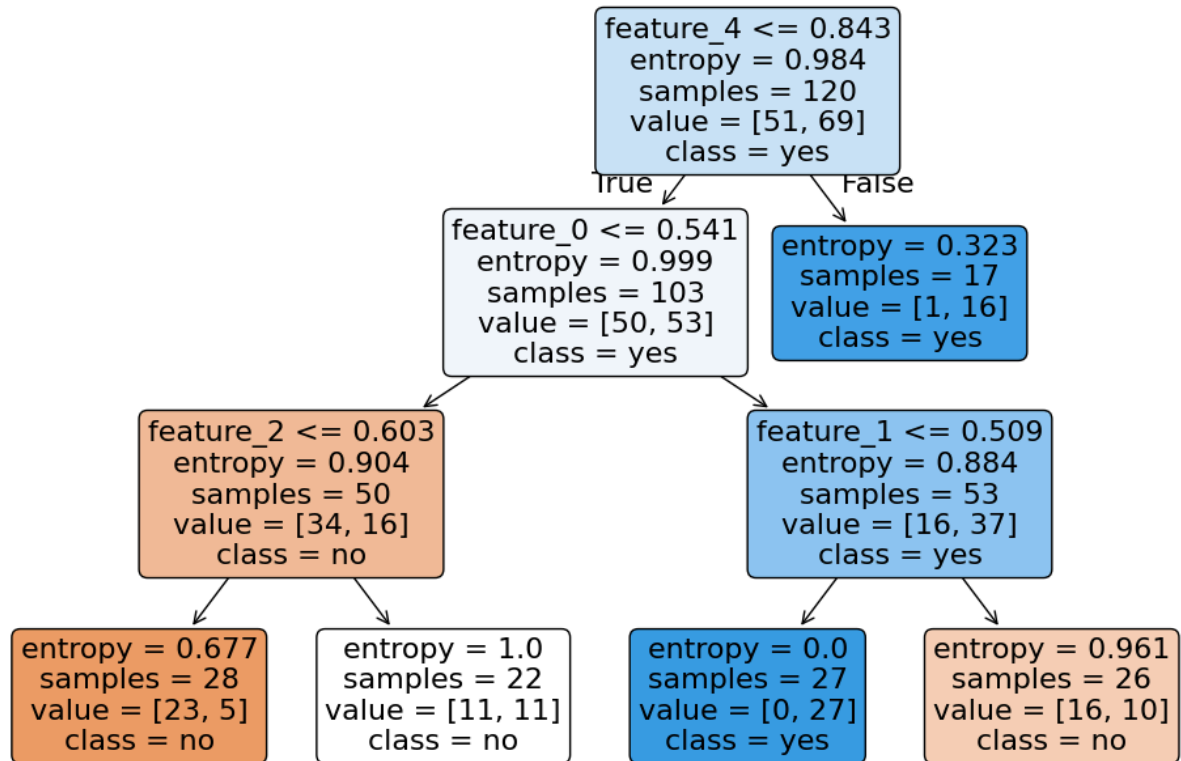
Accuracy: 0.77

Classification Report:

	precision	recall	f1-score	support
0	0.63	1.00	0.77	12
1	1.00	0.61	0.76	18
accuracy			0.77	30
macro avg	0.82	0.81	0.77	30
weighted avg	0.85	0.77	0.76	30

Confusion Matrix:

```
[[12  0]
 [ 7 11]]
```



```

In [152... clf = DecisionTreeClassifier(
    criterion='gini',
    min_samples_leaf= 20,
    random_state=42
)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()

```

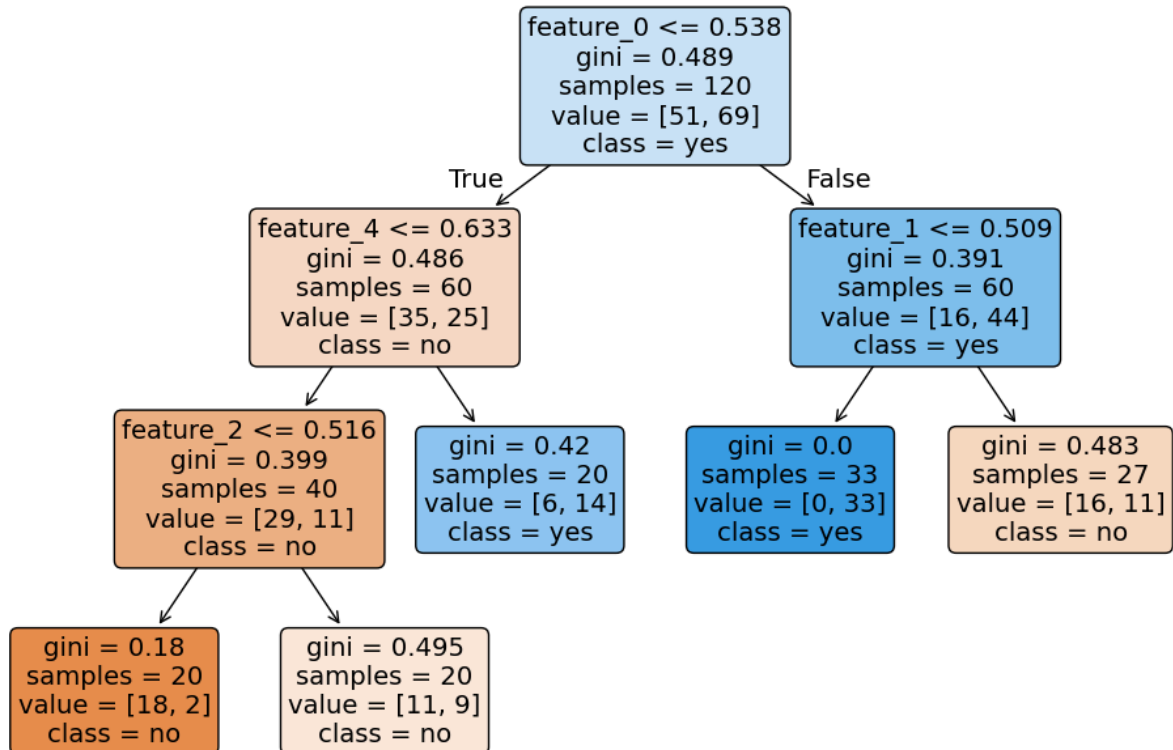
Accuracy: 0.80

Classification Report:

	precision	recall	f1-score	support
0	0.69	0.92	0.79	12
1	0.93	0.72	0.81	18
accuracy			0.80	30
macro avg	0.81	0.82	0.80	30
weighted avg	0.83	0.80	0.80	30

Confusion Matrix:

```
[[11  1]
 [ 5 13]]
```



Compare Gini Index vs Entropy

Answer: Gini Index ใช้สมการที่เป็น กำลัง 2 มีค่าอยู่ระหว่าง 0.0 ถึง 0.5 เหมาะกับข้อมูลขนาดใหญ่และมีมิติสูงกว่า

Entropy ใช้สมการที่เป็น logarithm มีค่าอยู่ระหว่าง 0.0 ถึง 1.0 เหมาะกับข้อมูลที่มีโครงสร้างและคลาสค่อนข้างสมดุล

Explain your understanding after trying these different parameters

Answer: max_depth จะทำให้ต้นไม้ที่ได้มีความลึกตามจำนวนเลขที่ใส่, min_samples_split จะทำให้ต้นไม้จะ split ตามจำนวนที่ใส่ถ้ามี sample มากกว่าเลขที่ใส่จะทำให้เกิดการ split, min_samples_leaf ทำให้แต่ละ leaves มี sample มากกว่าหรือเท่ากับจำนวนเลขที่เราใส่